

Formal Verification: Nova Linear Blockchain Mode

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize Nova, the simplest consensus mode that wraps Ray to provide linear blockchain semantics. Blocks form a chain with parent links, heights are strictly increasing, and the genesis block is always first.

1 Introduction

Nova is the linear blockchain consensus mode, composing as $\text{Nova} \rightarrow \text{Ray} \rightarrow \text{Wave} \rightarrow \text{Prism}$. It delegates all consensus logic to Ray, which delegates voting to Wave. This is the simplest operational mode for chains that need sequential block ordering.

2 Definitions

Definition 1 (Block). *A block with id, parent id, and height.*

Definition 2 (NovaState). *Chain state: list of blocks and last height.*

3 Theorems and Axioms

Theorem 3 (height_increases). *Block height is strictly increasing on append: $(\text{appendBlock } s \ b).\text{lastHeight} > s.\text{lastHeight}$.*

Proof. By omega on the height constraint. □

Theorem 4 (chain_grows). *Chain length increases by 1 on append.*

Theorem 5 (genesis_height). *Genesis block has height 0.*

Theorem 6 (genesis_single). *Genesis chain has exactly one block.*

4 Lean Source

```
/-  
  Nova Protocol Formal Model  
  
  Models protocol/nova/nova.go -- linear blockchain consensus mode.  
  
  Nova wraps Ray (linear decision engine) and provides blockchain semantics.
```

It is the simplest consensus mode: blocks form a chain, decided sequentially

Maps to:

- nova.go: Nova struct wrapping ray.Driver
- nova.go: Config{SampleSize, Alpha, Beta, RoundTO, GenesisHash}
- nova.go: NewNova, Start, Stop, Status

*Nova delegates all consensus logic to Ray, which delegates voting to Wave.
The composition: Nova -> Ray -> Wave -> Prism (sampling)*

Properties:

- Chain property: decided blocks form a valid chain (parent links)
- Genesis: genesis block is always the first decided block
- Height monotonicity: block heights are strictly increasing

-/

```
import Mathlib.Data.Nat.Defs
import Mathlib.Tactic

namespace Protocol.Nova

/-- Block in linear chain -/
structure Block where
  id      : Nat
  parentId : Nat
  height  : Nat

/-- Nova chain state -/
structure NovaState where
  chain      : List Block
  lastHeight : Nat

/-- Genesis state -/
def genesis (genesisBlock : Block) (hg : genesisBlock.height = 0) : NovaState
:=
{ chain := [genesisBlock], lastHeight := 0 }

/-- Append block to chain -/
def appendBlock (s : NovaState) (b : Block)
  (hparent : b.parentId = (s.chain.head?.map Block.id).getD 0)
  (hheight : b.height = s.lastHeight + 1) : NovaState :=
{ chain := b :: s.chain, lastHeight := b.height }

/-- Height is strictly increasing -/
theorem height_increases (s : NovaState) (b : Block)
  (hparent : b.parentId = (s.chain.head?.map Block.id).getD 0)
  (hheight : b.height = s.lastHeight + 1) :
  (appendBlock s b hparent hheight).lastHeight > s.lastHeight := by
simp [appendBlock, hheight]
omega

/-- Chain grows on append -/
theorem chain_grows (s : NovaState) (b : Block)
```

```

      (hp : b.parentId = (s.chain.head?.map Block.id).getD 0)
      (hh : b.height = s.lastHeight + 1) :
      (appendBlock s b hp hh).chain.length = s.chain.length + 1 := by
simp [appendBlock, List.length_cons]

/-- Genesis has height 0 -/
theorem genesis_height (b : Block) (hg : b.height = 0) :
  (genesis b hg).lastHeight = 0 := rfl

/-- Genesis chain has exactly one block -/
theorem genesis_single (b : Block) (hg : b.height = 0) :
  (genesis b hg).chain.length = 1 := rfl

end Protocol.Nova

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Protocol/Nova.lean>

White papers: <https://papers.lux.network>

Related white papers:

- `lux-consensus.tex`